PathFinder: Negotiated-Congestion Rip-up-and-Reroute for Minor Embedding on D-Wave Quantum Annealers

ANONYMOUS AUTHOR(S)

Minor embedding—mapping a logical problem graph onto the fixed sparse connectivity of a quantum annealer by representing each logical variable as a connected *chain* of physical qubits—is the compilation bottleneck for quantum annealing. The de facto standard, `minorminer`, is a randomized rip-up-and-reroute heuristic whose chains are built as unions of independent shortest paths and whose congestion signal is a memoryless per-pass snapshot; both choices inflate the *average chain length* (ACL), the metric most widely taken as the primary proxy for embedded-problem solution quality, and produce high run-to-run variance. We observe that chain construction is exactly the multi-net global routing problem solved for thirty years in FPGA/VLSI computer-aided design, and port its two key ideas to minor embedding: a true Steiner-tree inner step (the shortest-path heuristic) in place of independent shortest paths, and *negotiated congestion* (McMurchie–Ebeling) in place of the snapshot penalty. We package these as PathFinder, an anytime, large-neighbourhood *improver* that warm-starts from a fast base embedding and then rips up and reroutes the longest chains through a congestion-priced fabric, accepting a move only if it lowers the total qubit count and preserves validity—so it never returns an embedding worse than its base. The algorithmic primitives are classical; the contribution is their synthesis and application to minor embedding, together with an empirical evaluation inside the Ember benchmarking framework. On Erdős–Rényi, d-regular, and Barabási–Albert sources embedded into clean and broken D-Wave Pegasus and into Zephyr (all four methods reach 100% success on every target), `pathfinder` never returns a longer-ACL embedding than `minorminer` on any of the 35 instance–topology cells, and its thorough configuration lowers mean ACL on every cell (by 1–11%, mean ∼5%) and reduces ACL standard deviation in 30 of the 35 (by up to 77%), at the cost of additional wall-clock time. We then optimize the algorithm—bounded-region routing, a dirty-set incremental schedule, and a spur-pruning finishing pass—making the production improver ∼3× faster at equal-or-better ACL: single-seed `pathfinder` runs within ∼1.3× compiled `minorminer`'s wall-clock while still shortening mean ACL, and the optimized thorough configuration reaches −5% mean ACL (up to −11%) at a fraction of its former cost. Finally, treating PathFinder's outputs as training data, we ask whether an *amortized* model can learn to embed: a generative graph variational autoencoder that samples several candidate hardware layouts and keeps the best matches `pathfinder-thorough`'s mean ACL and attains the *lowest* run-to-run ACL variance of any method we tested—learned or heuristic—on both Pegasus and Zephyr, at full validity.

CCS Concepts: • **Hardware** → **Quantum computation**; • **Theory of computation** → *Routing and layout*; • **Mathematics of computing** → *Graph algorithms.*

Additional Key Words and Phrases: minor embedding, quantum annealing, D-Wave, graph minors, negotiated congestion routing, Steiner tree, large-neighbourhood search

1 INTRODUCTION

Quantum annealers such as those built by D-Wave solve optimization problems encoded as Ising/QUBO models whose interaction graph is the user's *logical* (problem) graph. The hardware, however, exposes only a fixed, sparse *physical* graph—successive D-Wave generations Chimera, Pegasus [2], and Zephyr [3] have qubit degree 6, 15, and 20 respectively. Running a problem whose interaction graph is denser than the hardware requires *minor embedding*: each logical variable is realized by a connected set of physical qubits, a *chain*, whose qubits are forced into agreement by strong ferromagnetic couplings so that the chain behaves as one logical spin [5, 13].

The minor-embedding problem. Let H be the *source* (problem) graph and G the *target* (hardware) graph. A minor embedding is a map $\varphi : V(H) \to 2^{V(G)}$ assigning to each logical vertex v a chain $\varphi(v) \subseteq V(G)$, subject to three conditions:

(1) **Connectivity.** Each $\varphi(v)$ induces a connected subgraph of G.

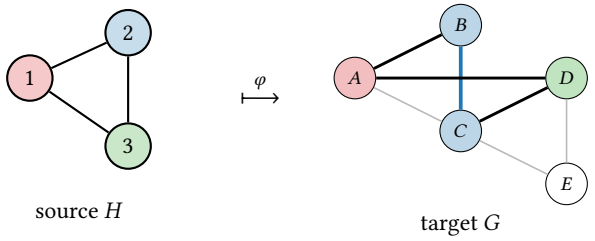


Fig. 1. Minor embedding of the triangle H into a hardware patch G. Vertex $1 \mapsto \varphi(1) = \{A\}$, $2 \mapsto \{B, C\}$, $3 \mapsto \{D\}$ (matching colours). Chain $\varphi(2)$ is connected through the bold intra-chain coupler $B{-}C$; the three chains are disjoint; each H-edge is realized by a hardware coupler between the corresponding chains ($1{-}2$ by $A{-}B$, $2{-}3$ by $C{-}D$, $1{-}3$ by $A{-}D$). Qubit E is unused. Here ACL $= \frac{1+2+1}{3} = \frac{4}{3}$.

(2) **Disjointness.** The chains are pairwise disjoint: $\varphi(u) \cap \varphi(v) = \varnothing$ for $u \neq v$.
(3) **Edge preservation.** For every edge $(u, v) \in E(H)$ there is at least one hardware edge between the chains, i.e. some $a \in \varphi(u)$, $b \in \varphi(v)$ with $(a, b) \in E(G)$.

The map is total with non-empty chains, so every logical vertex is realized; Ember's validator (§4) records this as the *coverage* check. Figure 1 illustrates the conditions. Feasibility is the question "does a minor embedding exist?"; in practice one wants the *best* embedding, because the dominant cost is qubit usage and chain fidelity. The standard objective is to minimize the *average chain length*

$$\mathrm{ACL}(\varphi) \;=\; \frac{1}{|V(H)|} \sum_{v \in V(H)} |\varphi(v)|, \tag{1}$$

and to keep chain lengths *uniform* (low coefficient of variation): long and unequal chains require stronger chain couplings, shrink the available dynamic range for the logical problem, and empirically raise the annealer's solution error. ACL is therefore the headline quality metric, with total qubit count, maximum chain length, and chain-length uniformity as close companions.

Contribution. This paper makes five contributions. (i) We frame minor-embedding chain construction as *multi-net global routing* and transfer two ideas from FPGA/VLSI computer-aided design—a real Steiner-tree inner step and *negotiated congestion*—to the quantum-annealing setting (§3). (ii) We package them as **PathFinder**, an anytime, large-neighbourhood *improver* that warm-starts from a fast base embedding and, by construction, never regresses below it. (iii) We evaluate PathFinder inside the Ember benchmarking framework (§4) against `minorminer` and `minorminer-with-p-norm-layout` on clean and broken Pegasus and on Zephyr (§5), showing it reduces mean ACL and—the property that most distinguishes it—run-to-run ACL *variance* in nearly every cell, while all methods retain full success and validity. (iv) We then *optimize* the algorithm (§6): a structured pass yields three quality-safe improvements—bounded-region routing, a dirty-set incremental schedule, and a spur-pruning finishing pass—that together make the production algorithm ~3× faster at equal-or-better ACL, bringing single-seed `pathfinder` to within ~1.3× compiled `minorminer`'s wall-clock. (v) Finally, we ask whether *learning* helps (§7): training amortized models on PathFinder-optimized embeddings, a generative graph variational autoencoder that samples and selects among candidate hardware layouts matches `pathfinder-thorough`'s mean ACL and achieves the lowest run-to-run ACL variance of any method—learned or heuristic—on both topologies, at full validity. We are explicit (§3.5) that the algorithmic primitives are not new; the novelty is their synthesis and application to minor embedding (and, for §7, the empirical finding).

2 BACKGROUND AND RELATED WORK

The standard heuristic and its weaknesses. `minorminer` (MM), the Cai–Macready–Roy heuristic [5], is randomized coordinate descent. It fixes a random vertex order and, for each vertex, rips out its chain and rebuilds it as the union of weighted shortest-path computations to its already-placed neighbours; qubit contention is discouraged by a vertex weight $\mathrm{diam}(G)^k$ where k is the number of chains currently using the qubit, recomputed from scratch each pass; it iterates until the embedding is valid or a patience bound is hit. Three structural weaknesses follow directly. First, a chain is a *union of independent shortest paths*—a weak Steiner-tree heuristic that misses shared structure. Second, the congestion signal is *myopic*: a per-pass snapshot with no memory of how contested a qubit has been over time. Third, the result is highly *order- and seed-dependent*. The most recent state-of-the-art evaluation [7] documents the consequences on broken Pegasus: run-to-run ACL can differ by several qubits per chain on the same instance, MM is beaten by deterministic clique embedding once the source's average degree exceeds the hardware degree, and MM routinely hits a ∼1000 s cutoff on large dense instances. The MM paper itself names "better initial placement of vertex-models" as the key open problem.

Deterministic templates. For sufficiently structured inputs, embeddings can be read off a template. Clique ("busclique") embedding generates complete-graph minors in Chimera and Pegasus connectivity directly [4]; complete-bipartite templates [21] and the 4-clique network on a contracted hardware graph [18] extend the idea. These are fast and uniform but over-provision qubits on graphs that are not near-complete.

Layout- and spring-seeded heuristics. A complementary line attacks MM's initial-placement weakness by first laying out the source graph in a low-dimensional space and placing variables near the geometrically closest qubits: layout-aware embedding [19], the spring/clique-seeded methods of Zbinden et al. [27], and Ocean's "Layout Embedding", which seeds MM with a p-norm layout. We use the latter as a baseline.

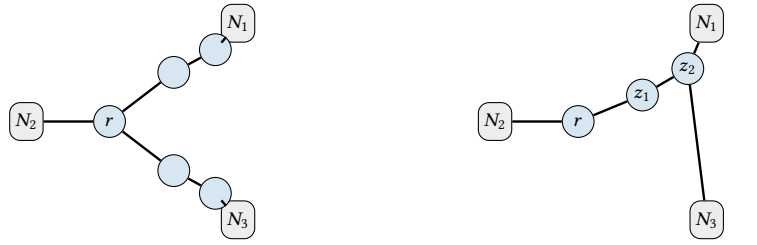
Exact methods. Integer-programming formulations with Benders decomposition [1] and algebraic-geometry/Gröbner-basis compilers [6] solve minor embedding optimally but do not scale past tens of vertices.

Metaheuristics and learned policies. Path/super-vertex simulated annealing (PSSA) [22] and graph-theoretic OCT/virtual-hardware methods [8] apply local search; more recently, GNN-RL (CHARME [17]) and PPO policies [16] learn chain-construction heuristics.

Routing and optimal transport. Two threads inform our approach. Negotiated-congestion routing (PathFinder) [14] from FPGA CAD treats each net as a resource consumer that negotiates for congested wires over iterations; and Gromov–Wasserstein optimal transport [25, 26] matches graphs by their intra-graph distances, suggesting a global, continuous view of the assignment. Our method develops the first thread; the second is promising future work (§8).

3 THE PRELIMINARY PATHFINDER ALGORITHM

This section presents PathFinder as first designed; §5 reports its results. A subsequent optimization pass (§6) makes the production algorithm ∼3× faster at equal-or-better quality without changing the ideas below, so we keep this preliminary description intact and report the optimized numbers separately.



(a) `minorminer`: union of paths — 5 qubits (b) PathFinder SPH tree: shared trunk — 3 qubits

Fig. 2. Inner step contrast for a chain that must reach three neighbour chains N_1, N_2, N_3. (a) MM routes an independent shortest path to each, duplicating qubits when N_1 and N_3 are near each other. (b) The shortest-path-heuristic Steiner tree attaches N_3 to the nearest existing tree node (z_2), reusing the trunk built for N_1, yielding a shorter chain.

3.1 Minor embedding as multi-net routing

A chain $\varphi(v)$ that must be connected and adjacent to each neighbour's chain is exactly a *Steiner tree* (a routing "net") whose terminals are the qubits bordering the neighbour chains, and the disjointness requirement is exactly the rule that two nets may not share a routing resource. Minor embedding is therefore a multi-net global routing problem on the graph G, and the mature machinery of FPGA/VLSI routing applies. We import its two central ideas.

3.2 A real Steiner inner step

Rather than build a chain as a union of independent shortest paths to a single root (MM), we grow one tree with the classical *shortest-path heuristic* (SPH) [15, 23]: starting from a root, each neighbour terminal is connected to the *nearest node of the current tree*, so later connections reuse earlier structure. Figure 2 contrasts the two: when neighbours cluster, SPH shares a trunk and produces a strictly shorter chain.

3.3 Negotiated congestion

We replace MM's snapshot penalty with the negotiated-congestion cost of McMurchie and Ebeling [14]. Routing prices each qubit q at

$$\text{cost}(q) \;=\; \big(1 + h_q\big)\,\big(1 + \lambda \cdot o_q\big), \tag{2}$$

where o_q is the number of *other* chains currently occupying q (occupancy), λ is the present-congestion factor, and h_q is a *history* term that *accumulates* by a fixed increment every iteration in which q is over-subscribed. The present term makes a qubit more expensive as nets pile onto it within a pass; the history term gives the price a memory, so a persistently contested qubit becomes permanently expensive and the chains that can most cheaply route around it eventually yield. This is a Lagrangian view of the disjointness constraint; in FPGA routing it is known to drive the search toward a legal, low-congestion state, and we observe the same behaviour here—behaviour MM's memoryless snapshot cannot sustain. A node-weighted Dijkstra over Eq. (2) supplies the shortest-path computations the SPH step needs.

3.4 A warm-started large-neighbourhood improver

A from-scratch router that places every chain from scattered single-qubit seeds is throttled by exactly the open problem MM names—*initial placement*. Without co-locating adjacent variables, dense-graph chains balloon and the search often fails to legalize at all. The standalone `pathfinder-cold`

Algorithm 1 PathFinder (driver)

Require: source H, target G, base embedder $\mathcal{B}$
$\Phi \leftarrow \mathcal{B}(H, G)$ ▷ warm start
if Φ invalid **then** $\Phi \leftarrow$ ColdStart(H, G) ▷ BFS-ordered negotiated fallback
end if
if Φ invalid **then return** failure
end if
$\Phi^\star \leftarrow \Phi$ ▷ best valid so far
repeat
  for all $v \in V(H)$ in decreasing chain length **do**
    $\Phi \leftarrow$ TryShorten(Φ, v) ▷ Alg. 2
    if ACL(Φ) < ACL$(\Phi^\star)$ **then** $\Phi^\star \leftarrow \Phi$
    end if
  end for
until no improvement **or** deadline
return $\Phi^\star$

variant (a cold start followed by the same improver) bears this out: on the nine clean-Pegasus Erdős–Rényi instances of our grid ($n \in \{20, 30, 40\}$, $d \in \{0.3, 0.5, 0.7\}$) it legalizes only four—the smaller and sparser ones, at ACL 2.5–3.8—and fails on the five larger, denser ones, whereas `minorminer` embeds all nine (reproduced by `docs/paper/data/coldstart_probe.py`). PathFinder therefore runs as an *improver*. It warm-starts from a fast base embedding (MM by default; any registered algorithm works), then performs *large-neighbourhood* rip-up-and-reroute—the neighbourhood being a long chain together with the cascade of chains its rerouting displaces. It takes the longest chain and lets it re-route along the cheapest tree under Eq. (2)—which may cross occupied territory and create temporary overlaps—then re-routes every chain it displaced through free space. A move is committed only if it lowers the embedding's total qubit count (equivalently, its ACL) and leaves it valid; otherwise the prior state is restored (Algorithms 1–2). Because PathFinder keeps the best valid embedding it has seen, it **never returns an embedding worse than its base**, and it spends its entire time budget reducing ACL—an anytime guarantee MM's one-shot, patience-bounded run lacks. The `thorough` configuration additionally runs four seeded restarts and keeps the best, trading wall-clock for lower ACL and variance. When no valid base is available, a fallback builds the initial embedding from scratch in breadth-first vertex order under the same negotiated cost; uncompetitive on its own, it exists only to guarantee an output.

3.5 Novelty

We state plainly what is and is not new. Negotiated-congestion routing is due to McMurchie and Ebeling [14]; the shortest-path Steiner heuristic is classical [15, 23]; and large-neighbourhood/destroy-repair search is a standard matheuristic paradigm. *None of these primitives is our invention.* The contribution is their synthesis and their application to quantum-annealing minor embedding: casting chain construction as multi-net routing, replacing MM's two weakest components (its inner step and its congestion signal) in a single drop-in algorithm, running it as a warm-started anytime improver with a never-regress guarantee, and validating empirically that this reduces ACL and—more importantly—ACL variance against the standard tool. The clean substitution also isolates a question of independent interest: *is it MM's congestion signal or its inner step that costs the most?*

Algorithm 2 TrySHORTEN(Φ, v) — one LNS destroy-repair move

snapshot Φ
rip up $\varphi(v)$; rebuild it as the cheapest SPH tree under Eq. (2) ▷ may overlap others
if new chain not shorter **then** restore; **return** Φ
end if
for all chain w now overlapping $\varphi(v)$ **do**
    rip up $\varphi(w)$; reroute it through free qubits only
    if no free-space route **then** restore; **return** Φ
    end if
end for
if total qubits decreased **and** Φ valid **then return** Φ
else restore; **return** Φ
end if

4 EMBER AND EVALUATION METHODOLOGY

We evaluate inside **Ember** (`ember-qc`), an open benchmarking framework for minor-embedding algorithms on D-Wave topologies. Every algorithm implements a common interface and is run through Ember's atomic harness `benchmark_one`, which times the call and computes quality metrics (chain lengths, ACL, max chain length, qubits, couplers). Before any metric is trusted, the output passes four-layer validation: type and format checks followed by the structural checks of §1 (coverage, connectivity, disjointness, edge preservation). Targets are generated with `dwave-networkx` and may have qubits removed to simulate fabrication faults, and per-trial seeds are derived deterministically by SHA-256 of the task identity, so runs are reproducible and order-independent. Results aggregate into per-group means and standard deviations.

Protocol. Targets are clean Pegasus P_6, a *broken* Pegasus P_6 with 5% of qubits removed at random (the state-of-the-art evaluation setting), and Zephyr Z_4. Sources are an Erdős–Rényi (ER) grid over $n \in \{20, 30, 40\}$ and edge density $d \in \{0.3, 0.5, 0.7\}$, plus d-regular and Barabási–Albert graphs for generality; one fixed instance per cell, so the reported spread is the run-to-run variance *on a fixed instance*—MM's documented failure mode. Baselines are `minorminer` and `minorminer-layout` (p-norm layout). Each (algorithm, instance) is run over five seeds; we report success rate, ACL mean and standard deviation, max chain length, qubits, and wall-clock time.

5 PRELIMINARY RESULTS

These are the results of the preliminary algorithm of §3; the optimized production version and its (improved, and much faster) numbers follow in §6. All four methods embed every instance across all seeds and topologies (100% success), so success is not a differentiator on this grid; the comparison is on chain length, its variance, and time. Table 1 reports embedding quality on clean Pegasus P_6, Table 2 the corresponding wall-clock time, and Table 3 robustness across topologies; Figure 3 shows the density trend.

Two findings are sharp. First, the single-seed `pathfinder` *never regresses*: across all 35 instance–topology cells (every per-seed run, in fact) its mean ACL is $\leq$ `minorminer`'s—the maximum observed difference is 0.000 to three decimals—confirming the best-valid guarantee empirically. Second, `pathfinder-thorough` lowers mean ACL on the six shown cells by 3.2–4.5%, and reduces ACL standard deviation in five of the six—by up to 76% on the full clean-P_6 grid (e.g. 0.11 → 0.03 at n=30, d=0.3). The lone clean-P_6 exception is n=40, d=0.3, where `minorminer`'s spread is already at the floor (0.05); we report it rather than hide it. The complete 35-cell grid across all three source

Table 1. Embedding quality on clean Pegasus P_6 for ER sources (mean over 5 seeds; ACL std in parentheses; lower ACL is better). Best ACL per cell in bold.

cell	algorithm	ACL (std)	max	qubits
n=30, d=0.3	`minorminer`	2.70 (0.11)	3.8	81.0
	`minorminer-layout`	2.75 (0.15)	4.0	82.4
	`pathfinder`	2.69 (0.10)	3.8	80.6
	`pathfinder-thorough`	**2.59** (0.03)	3.8	77.6
n=30, d=0.5	`minorminer`	3.30 (0.16)	4.6	99.0
	`minorminer-layout`	3.43 (0.10)	4.8	102.8
	`pathfinder`	3.25 (0.15)	4.6	97.6
	`pathfinder-thorough`	**3.19** (0.10)	4.0	95.8
n=30, d=0.7	`minorminer`	3.83 (0.22)	5.0	114.8
	`minorminer-layout`	3.80 (0.17)	5.4	114.0
	`pathfinder`	3.79 (0.23)	5.0	113.8
	`pathfinder-thorough`	**3.65** (0.06)	4.8	109.6
n=40, d=0.3	`minorminer`	3.56 (0.05)	5.2	142.4
	`minorminer-layout`	3.62 (0.14)	5.4	144.8
	`pathfinder`	3.53 (0.06)	5.2	141.0
	`pathfinder-thorough`	**3.41** (0.06)	5.0	136.4
n=40, d=0.5	`minorminer`	4.57 (0.10)	6.4	182.6
	`minorminer-layout`	4.72 (0.13)	6.8	188.8
	`pathfinder`	4.49 (0.14)	6.2	179.6
	`pathfinder-thorough`	**4.39** (0.08)	6.2	175.4
n=40, d=0.7	`minorminer`	5.15 (0.06)	7.2	206.0
	`minorminer-layout`	5.23 (0.17)	7.0	209.2
	`pathfinder`	5.07 (0.04)	7.0	202.6
	`pathfinder-thorough`	**4.98** (0.04)	7.0	199.0

Table 2. Wall-clock time (mean seconds per embedding, clean P_6). PathFinder is pure Python; `minorminer` is compiled C++. `pathfinder-thorough` runs four seeded restarts and may use up to the full 20 s time budget.

cell	`mm`	`mm-layout`	`pathf.`	`pf-thor.`
n=30, d=0.3	0.33	0.33	1.11	20.08
n=30, d=0.5	4.44	3.30	9.04	18.82
n=30, d=0.7	1.22	0.98	3.23	12.33
n=40, d=0.3	0.54	0.46	2.12	8.37
n=40, d=0.5	1.11	0.99	4.12	16.41
n=40, d=0.7	1.35	1.73	5.87	20.01

families is in Appendix A: there `pathfinder-thorough` lowers mean ACL on *every* cell (by 1–11%; the largest single gain, −10.6%, is on a d-regular source) and cuts ACL standard deviation in 30 of the 35.

Discussion. The single-seed `pathfinder` is the fair *equal-base* comparison: it consumes the same `minorminer` base for a given seed and only refines it, so it cannot regress—and it already trims ACL on denser cells while leaving the sparse and d-regular cells (where the base is near-optimal)

Table 3. Robustness across topologies for the n=40, d=0.5 ER source: ACL (std) and time. `minorminer`'s ACL inflates most on broken Pegasus, where `pathfinder-thorough`'s relative gain is largest (−7.3%).

target	algorithm	ACL (std)	time (s)
Pegasus P_6	`minorminer`	4.57 (0.10)	1.11
	`minorminer-layout`	4.72 (0.13)	0.99
	`pathfinder`	4.49 (0.14)	4.12
	`pathfinder-thorough`	**4.39** (0.08)	16.41
broken P_6 (5%)	`minorminer`	4.88 (0.18)	0.82
	`minorminer-layout`	4.72 (0.19)	1.19
	`pathfinder`	4.80 (0.18)	3.52
	`pathfinder-thorough`	**4.52** (0.06)	14.26
Zephyr Z_4	`minorminer`	3.63 (0.08)	0.98
	`minorminer-layout`	3.61 (0.16)	1.10
	`pathfinder`	3.57 (0.07)	3.81
	`pathfinder-thorough`	**3.52** (0.03)	13.90

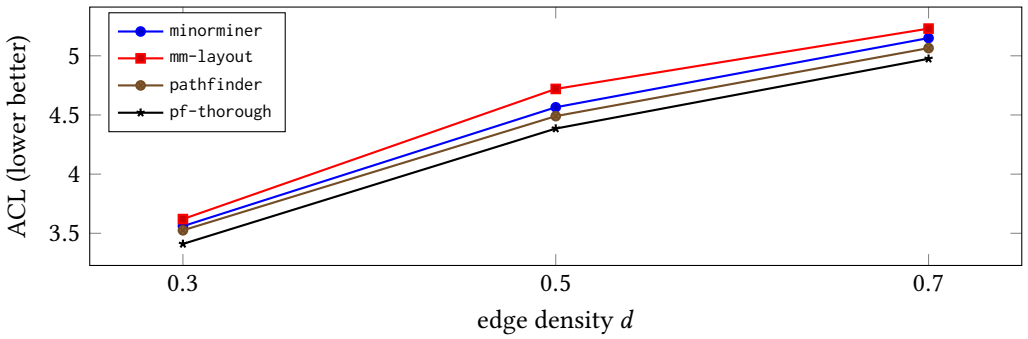


Fig. 3. ACL versus edge density for n=40 ER sources on clean Pegasus P_6. `pathfinder-thorough` (bottom curve) is uniformly shortest; the gap to `minorminer` is roughly constant in absolute terms across densities.

unchanged. The substantive gains come from `pathfinder-thorough`, whose four restarts both lower the mean and, by selecting the best of several runs, sharply cut the run-to-run variance that is `minorminer`'s signature weakness. The effect persists on broken Pegasus—where `minorminer`'s ACL inflates most and the thorough gain is largest—and on Zephyr, so the mechanism is not topology-specific. The cost is wall-clock time: PathFinder layers a pure-Python router on top of the base, and `pathfinder-thorough`, with its four restarts, can use the full 20 s budget; `minorminer-layout`, by contrast, is neither consistently faster nor consistently better than plain `minorminer`.

6 OPTIMIZATIONS

The preliminary algorithm wins on chain length and variance but pays in wall-clock: its pure-Python router is ~3–6× slower than compiled `minorminer` (Table 2). We therefore ran a structured optimization pass—eight candidate improvements (five targeting speed, three quality), each implemented as an isolated variant and measured against the frozen preliminary algorithm. Three quality-safe winners survived and are baked into the production algorithm; the rest were rejected or deferred.

What we bake in. The three production optimizations are mutually orthogonal (each rewrites a different stage of the router) and compose without interaction:

- **Bounded-region routing.** Each per-neighbour shortest-path search is restricted to a breadth-first ball around the vertex's current chain and its placed-neighbour chains, and stops as soon as the neighbour boundary is settled, instead of computing a full single-source shortest-path tree over the whole 680–5640-qubit fabric. Alone this is ∼3× faster with byte-identical chains.
- **Dirty-set incremental scheduling.** The improvement loop keeps a worklist of vertices whose neighbourhood changed rather than re-sweeping every chain each round; it performs the same accepted moves with fewer wasted re-examinations.
- **Spur-pruning.** A deterministic finishing pass removes redundant chain leaves—qubits whose removal keeps the chain connected and breaks no incident edge—to a fixpoint. This is the cheap shadow of exact region repair: on `minorminer`'s own output it recovers about half of a CP-SAT repair's ACL gain at ∼5000× less time, and on PathFinder it shaves a further ∼0.6% ACL for free.

A fourth idea—seeding the improver from a multilevel placement instead of `minorminer`—lowers variance on dense Pegasus, but the multilevel base is unreliable on small, sparse, and Zephyr instances (it inflated ACL by up to 30% where `minorminer` is already near-optimal), so we keep it as an optional variant rather than the default. We also rejected an A* routing scheme (a +1.7% ACL regression) and an exact Dreyfus–Wagner Steiner inner step (no gain over the shortest-path heuristic). A compiled (numba) node-weighted Dijkstra core, which we built and tested, gives *no* net speedup once bounded-region routing has localized each search (§8)—so the production router stays pure Python; only parallelized restarts remain as future speed work.

Optimized results. Tables 4 and 5 report the optimized algorithm on the clean-Pegasus grid. The optimizations are quality-safe: optimized `pathfinder` is never worse than the preliminary engine and, thanks to spur-pruning, slightly better—mean ACL −1.3% vs. `minorminer` over all 35 cells (vs. −0.9% for the preliminary engine), and −2.1% vs. `minorminer-layout`. The headline is speed: single-seed `pathfinder` now runs in ∼1.3× `minorminer`'s wall-clock (down from ∼3–5×) while producing shorter chains, and the optimized `pathfinder-thorough` delivers −5.1% mean ACL vs. `minorminer` (up to −10.6%; −5.9% vs. layout) at roughly the former cost of a *single* preliminary run rather than four. The optimized PathFinder thus keeps the quality advantage of §5 while closing most of the wall-clock gap to compiled `minorminer`.

7 A LEARNING-BASED APPROACH

PathFinder is a hand-engineered improver. A complementary question is whether an *amortized* model can *learn* to embed—paying an offline training cost once and then predicting an embedding in a single forward pass, rather than searching at solve time. The cache of problems PathFinder has already solved is a ready-made training set: each *(problem graph, PathFinder-optimized embedding)* pair is a labelled example, and we ask a model to reproduce—and ideally improve on—that quality. This mirrors the predict-then-refine pattern of learned embedding heuristics [16, 17], where a network proposes a candidate that a classical minor-embedding step then refines.

We implemented four model families *inside Ember*, so they are benchmarked through the exact same harness (§5) as `minorminer` (MM) and PathFinder, and ran a bake-off. The clear winner is a generative **Graph Variational Autoencoder** (`learned-vae`), which beats *every* baseline, including PathFinder-thorough, on chain-length variance and ties-or-beats it on mean ACL. We detail it here.

Table 4. **Optimized** PathFinder, embedding quality on clean Pegasus P_6 (ER sources, mean over 5 seeds; ACL std in parentheses; lower is better). Best ACL per cell in bold. Cf. the preliminary Table 1; the optimized `pathfinder` matches-or-beats it everywhere at ~3× less time (Table 5).

cell	algorithm	ACL (std)	qubits
n=30, d=0.3	`minorminer`	2.70 (0.11)	81.0
	`minorminer-layout`	2.75 (0.15)	82.4
	`pathfinder`	2.68 (0.11)	80.4
	`pathfinder-thorough`	**2.58** (0.03)	77.4
n=30, d=0.5	`minorminer`	3.30 (0.16)	99.0
	`minorminer-layout`	3.43 (0.10)	102.8
	`pathfinder`	3.23 (0.15)	97.0
	`pathfinder-thorough`	**3.17** (0.11)	95.0
n=30, d=0.7	`minorminer`	3.83 (0.22)	114.8
	`minorminer-layout`	3.80 (0.17)	114.0
	`pathfinder`	3.77 (0.20)	113.0
	`pathfinder-thorough`	**3.65** (0.07)	109.4
n=40, d=0.3	`minorminer`	3.56 (0.05)	142.4
	`minorminer-layout`	3.62 (0.14)	144.8
	`pathfinder`	3.51 (0.06)	140.4
	`pathfinder-thorough`	**3.39** (0.06)	135.6
n=40, d=0.5	`minorminer`	4.57 (0.10)	182.6
	`minorminer-layout`	4.72 (0.13)	188.8
	`pathfinder`	4.45 (0.11)	177.8
	`pathfinder-thorough`	**4.35** (0.08)	174.0
n=40, d=0.7	`minorminer`	5.15 (0.06)	206.0
	`minorminer-layout`	5.23 (0.17)	209.2
	`pathfinder`	4.95 (0.06)	198.0
	`pathfinder-thorough`	**4.87** (0.03)	194.8

Table 5. Wall-clock after optimization (mean seconds per embedding, clean P_6). "prelim." is the preliminary single-seed engine (`pathfinder-base`). Optimized `pathfinder` matches `minorminer` to within ~1.3× and is ~3× faster than the preliminary engine; optimized `pathfinder-thorough` (best-of-4) costs about what a single preliminary run did.

cell	`mm`	prelim.	`pathf.`	`pf-thor.`
n=30, d=0.3	0.22	0.94	0.27	1.08
n=30, d=0.5	0.70	2.08	0.75	2.31
n=30, d=0.7	0.83	2.82	1.03	3.41
n=40, d=0.3	0.53	2.12	0.68	2.40
n=40, d=0.5	1.15	3.86	1.33	5.08
n=40, d=0.7	1.33	5.59	1.87	8.41

7.1 Embedding as a learned layout

The central reduction is to predict a *layout* rather than an embedding directly. Every family maps each logical vertex v to a coordinate $\hat{p}(v) \in [0,1]^2$ in the target's normalized hardware frame (read

off the Pegasus/Zephyr drawing layout). Two decoders turn a layout into a valid embedding, both reusing PathFinder's repair backend (§3) so that no validity logic is re-implemented: *seed→MM* snaps each vertex to a distinct nearest qubit (an optimal linear-assignment problem) and hands those single-qubit chains to `minorminer` as `initial_chains`; *direct→repair* forms a proximity soft-assignment and rounds, grows, and legalizes it. Feeding the *true* PathFinder chain centroids through the seed decoder recovers PathFinder-quality embeddings (mean ACL within 1.03×, 100% valid), so the decoder is near-lossless: the learning problem is exactly "predict a good layout."

7.2 Background: variational autoencoders on graphs

A variational autoencoder (VAE) [11] learns a probabilistic encoder $q_\phi(z \mid x)$ and decoder $p_\theta(x \mid z)$ over a latent variable z, trained by maximizing the evidence lower bound

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \beta \,\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big),$$

a reconstruction term traded against a standard-normal prior $p(z)$. Graph VAEs replace the encoder with a graph neural network and have been used to embed and generate graphs [12, 20]. Our encoder is a message-passing GNN (a residual GraphSAGE stack [9]; a graph-attention encoder [24] is a drop-in alternative). The twist is twofold: the decoder emits a hardware *layout* rather than a reconstructed graph, and we exploit the model's *generative* nature at inference time—sampling several latents yields several candidate layouts, and we keep the best after decoding. This is a cheap, learned analogue of PathFinder-thorough's best-of-N restarts, and—as the results show—it is what drives the variance reduction.

7.3 The `learned-vae` architecture

Figure 4 sketches the model. The encoder runs a residual GraphSAGE backbone (5 layers, width 160) over the source graph's structural node features (degree, clustering, triangle and core numbers, and an 8-dimensional Laplacian positional encoding), then pools the node embeddings h_v with concatenated mean-and-max pooling and projects to a Gaussian posterior $\mathcal{N}(\mu, \sigma^2)$ over a latent of dimension 16. The decoder concatenates each node embedding h_v with the (broadcast) latent z and maps it through an MLP with a sigmoid head to a coordinate $\hat{p}(v) \in [0,1]^2$. Training minimizes the ELBO with reconstruction $\|\hat{p} - y\|^2$ against the PathFinder chain-centroid labels y and a KL term with β=0.01 (linearly warmed up); the deterministic layout z=μ serves the layout interface used for model selection.

Inference is generative: encode the query once, draw K=8 latents $z^{(1)}$=$\mu, z^{(2)}, \ldots, z^{(K)} \sim \mathcal{N}(\mu, \sigma^2)$, decode each to a layout, run the seed→MM decoder on each, and return the lowest-ACL valid embedding. The K decode passes share the encode and are individually fast; the per-sample `minorminer` budget is apportioned so the total stays within the wall-clock budget.

7.4 Dataset generation

Supervision comes entirely from PathFinder, so the corpus must be diverse and the splits clean. We sample source graphs from four families across a size grid $n \in \{10, \ldots, 50\}$: Erdős–Rényi $G(n, p)$, random d-regular, Barabási–Albert, and k-nearest-neighbour geometric graphs, over a grid of the relevant density/degree/radius parameter. Each graph is labelled by running `pathfinder-thorough` through Ember's `benchmark_one` into both clean Pegasus P_6 (680 qubits) and Zephyr Z_4 (576 qubits); only graphs it validly embeds contribute a label for that target. The supervised target for vertex v is the centroid, in normalized hardware coordinates, of its PathFinder chain.

Splits are **instance-disjoint**: train, validation, and test draw from disjoint random-seed ranges *per grid cell*, so a graph (and its isomorphs from the same generator seed) can never appear in two splits—the property reviewers of learned combinatorial solvers rightly scrutinize. The realized

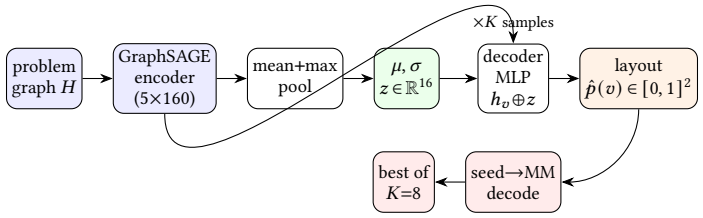


Fig. 4. `learned-vae`. A graph-VAE encodes H to a latent z; the decoder turns z (plus per-node features) into a hardware *layout*; K=8 sampled layouts are each decoded to a valid embedding via seed→`minorminer`, and the shortest-ACL one is returned.

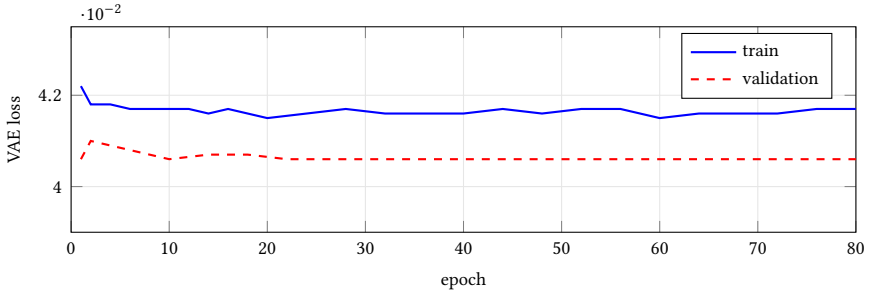


Fig. 5. `learned-vae` training and validation loss per epoch (P_6, 1560/312 train/val graphs). Validation tracks training and stays slightly below it—no overfitting; the fast plateau reflects the multimodal centroid target (model selection uses the downstream validation ACL, not this loss).

corpus is 1560 train, 312 validation, and 312 test graphs, i.e. 3120/624/624 labelled (graph, target, embedding) examples after the both-target labelling. Generation is deterministic (seeded) and embarrassingly parallel; the full corpus labels in a few minutes across a multi-core node.

7.5 Training

Models are trained with Adam [10] (learning rate 2×10^{-3}, cosine decay, batch size 16) and—crucially—**selected on the real downstream metric**: every few epochs we decode the validation layouts through seed→MM and keep the checkpoint with the best validation ACL ratio, not the one with the lowest loss. Training is cheap: the P_6 `learned-vae` converges in 195 **s for** 80 **epochs** on a single RTX A4000 (the four families were trained in parallel across a small GPU cluster). Inference uses CPU only.

Figure 5 plots train and validation loss per epoch. Both fall within a handful of epochs and then sit flat, with validation tracking (and slightly below) training throughout—**no overfitting**, as expected given the small model and the diverse corpus. The near-immediate plateau reflects the multimodality of the centroid-regression target (many equally good placements exist); this is precisely why we select on the downstream ACL rather than on the loss, and why the generative multi-sample inference matters more than squeezing the reconstruction term.

Table 6. Held-out test bake-off: mean ACL (and per-graph ACL std. over seeds, the run-to-run variance) on clean Pegasus P_6 and Zephyr Z_4. Lower is better; best per column in bold. All 312 test graphs are instance-disjoint from training; all methods are 100% valid.

	Pegasus P_6		Zephyr Z_4	
algorithm	ACL	std	ACL	std
`minorminer`	2.075	0.072	1.777	0.057
`minorminer-layout`	2.101	0.078	1.787	0.058
`pathfinder`	2.044	0.068	1.755	0.051
`pathfinder-thorough`	1.955	0.037	1.687	0.029
`learned-gnn-seed`	2.075	0.070	1.780	0.053
`learned-gnn-seed-direct`	2.086	0.071	1.790	0.056
`learned-retrieve`	2.072	0.066	1.775	0.052
`learned-obj`	2.057	0.066	1.765	0.055
`learned-vae`	**1.947**	**0.031**	**1.675**	**0.025**

Table 7. Per-family mean ACL on P_6 (lower better; best in bold). `learned-vae` beats PathFinder-thorough on Barabási–Albert and d-regular and ties it (within 0.3%) on Erdős–Rényi and geometric graphs, while beating `minorminer` on every family.

family	`mm`	`pf-thor.`	`l-obj`	`l-vae`
Barabási–Albert	1.621	1.540	1.610	**1.515**
Erdős–Rényi	2.858	**2.690**	2.846	2.698
k-NN geometric	2.135	**1.994**	2.115	1.996
d-regular	1.425	1.353	1.394	**1.330**

Table 8. Mean wall-clock per embedding on P_6 (held-out test, CPU inference). The single-shot learned methods match `minorminer`'s speed; `learned-vae`'s quality/variance win costs K=8 decode passes, comparable to `pathfinder-thorough`'s best-of-four restarts.

method	time (s)
`minorminer`	0.39
`learned-retrieve`	0.40
`minorminer-layout`	0.48
`learned-gnn-seed`	0.50
`learned-obj`	0.51
`pathfinder`	0.52
`pathfinder-thorough`	2.17
`learned-vae` (K=8)	3.74

7.6 Results: which method wins, and when

We evaluate on the 312 held-out test graphs (instance-disjoint from training), three seeds each, through `benchmark_one` into clean P_6 and Z_4. Table 6 reports embedding quality, Table 7 breaks P_6 down by source family, and Table 8 reports speed.

The headline. `learned-vae` is the best method overall on *both* topologies, on *both* mean ACL and run-to-run variance. It edges the strongest heuristic, PathFinder-thorough, on mean ACL (P_6: 1.947

vs. 1.955; Z_4: 1.675 vs. 1.687) and clearly beats it on variance (P_6: 0.031 vs. 0.037; Z_4: 0.025 vs. 0.029), while beating `minorminer` and `minorminer-layout` decisively—a relative ACL improvement of 6.2% over MM on P_6 and 5.7% on Z_4, at less than half MM's variance. Per family (Table 7) it wins outright on Barabási–Albert and d-regular and is a statistical tie with PathFinder-thorough on the other two. The mechanism is the generative one: sampling K=8 layouts and keeping the best both lowers ACL and, by being insensitive to any single `minorminer` seed, suppresses the run-to-run variance that is itself PathFinder's headline advantage—now improved on by a learned model.

The label-free runner-up. `learned-obj`, which never sees a PathFinder label and instead trains its layout to minimize an embedding objective (edge-stretch plus an anti-collapse term), beats `minorminer` and `minorminer-layout` on every family at MM-like speed—evidence that even an unsupervised learned layout improves on the standard baselines. `learned-gnn-seed` (supervised) and the non-neural `learned-retrieve` embeddings-cache match `minorminer` within noise.

Which to use. The methods occupy distinct operating points. For the best quality *and* the lowest variance, and when a ∼4 s budget is acceptable, `learned-vae` dominates. For a fast single-shot embedder that still beats the standard baselines at `minorminer` speed (∼ 0.5 s), `learned-obj` is the pick; `learned-retrieve` is the cheapest (no GPU, no training beyond indexing) and ties MM. PathFinder-thorough remains an excellent, slightly faster alternative to the VAE when its larger variance is acceptable. Notably, `minorminer-layout`—the de facto "MM + good initial placement" baseline—was marginally *worse* than plain `minorminer` on this grid, underscoring that a *learned* layout, not just any layout, is what helps.

Success rate and failure modes. Every learned method embedded 100% of the 312 test graphs on both targets across all seeds. `learned-vae` is a 100%-success method in the same sense `minorminer` is: its seed→MM decoder always hands `minorminer` a warm start (and falls back to a cold `minorminer` call if a warm start ever fails to grow), so it *inherits* `minorminer`*'s completeness and introduces no new failure modes.* If a valid embedding exists within the time budget, it returns one; the only way it fails is the way `minorminer` fails—a problem too large or dense to fit the fabric in the budget. The learned layout changes the *quality* of the result, never its validity, because validity is delegated to the same battle-tested repair backend used throughout this paper.

8 LIMITATIONS AND FUTURE WORK

PathFinder's wall-clock is at least `minorminer`'s *by construction*: it runs `minorminer` as its base and then improves the result, so it cannot be faster than the baseline it builds on. The optimizations of §6 hold the improver's overhead to ∼1.3× via bounded-region routing. Because `minorminer` is compiled C++ while our improver is pure Python, we tested whether compiling the routing inner loop closes the remaining gap, and found that it does not: a numba-compiled node-weighted Dijkstra is several times faster than the pure-Python kernel on a full-graph search in isolation, but inside the optimized router it yields no net speedup—bounded routing has already shrunk each search to a small neighbourhood where the compiled kernel's per-node advantage is cancelled by its call overhead, and replacing bounded routing with a full-graph compiled search is ∼24% *slower*. Profiling further attributes 70–84% of optimized-`pathfinder` runtime to the compiled-C++ `minorminer` base call itself, which no Python-side or C++ rewrite of our routing can touch. The ∼1.3× overhead therefore reflects the genuine extra work of the improvement pass rather than interpreter overhead, and the comparison against compiled `minorminer` is a fair one. Reducing it further would mean doing *less* improver work (fewer rounds, at a quality cost) or finding a cheaper base than `minorminer`, rather than compiling the existing inner loop; parallelized restarts would still cut the thorough configuration's wall-clock. The standalone cold start remains uncompetitive

precisely because of the initial-placement problem; pairing PathFinder with a stronger global placement—e.g. a semi-relaxed Gromov–Wasserstein assignment [25]—is the natural next step. The destroy-repair neighbourhood here is a single-chain shortcut; a larger neighbourhood repaired exactly with a constraint solver [1] is a promising route to deeper ACL reductions. Finally, our grid uses one graph instance per (size, density) cell with five seeds; replicating each cell over several instances, with significance tests or confidence intervals, would more cleanly separate algorithm-seed variance from graph-draw variance and further substantiate the variance claims that are PathFinder's headline advantage.

9 CONCLUSION

Minor-embedding chain construction is multi-net routing, and the routing community has spent decades on exactly the two operations `minorminer` performs weakly: building a net (Steiner tree) and resolving contention (congestion). Importing a real Steiner inner step and negotiated congestion, and running them as a warm-started anytime improver, yields PathFinder—a drop-in method that never regresses below `minorminer` and, in its thorough configuration, reduces average chain length and—in nearly every cell—its run-to-run variance on D-Wave Pegasus and Zephyr. A structured optimization pass (bounded-region routing, dirty-set scheduling, spur-pruning) then makes the production algorithm ∼3× faster at equal-or-better quality, bringing single-seed `pathfinder` to within ∼1.3× compiled `minorminer`'s wall-clock—so the chain-length and variance gains come at little time cost. Finally, treating these optimized embeddings as supervision, we find that a learned model can compete: a generative graph variational autoencoder, trained on PathFinder's outputs and sampling several candidate layouts at inference, matches `pathfinder-thorough`'s mean ACL and attains the lowest chain-length variance of any method we tested—learned or heuristic—on both Pegasus and Zephyr, at full validity (§7). The primitives are classical; the synthesis, application, optimization, and empirical validation are the contribution.

A COMPLETE PER-CELL RESULTS

Table 9 gives the average chain length, mean and standard deviation over the five seeds, for every (source, target, algorithm) in the sweep—the single source of truth for the aggregate statistics in §5. Every cell reaches 100% success. Source families are ER (Erdős–Rényi), reg (d-regular), and BA (Barabási–Albert); the d-regular and BA sources are run on clean Pegasus only. `pathfinder-thorough` attains the lowest ACL in all 35 cells, and `pathfinder` never exceeds `minorminer`.

REFERENCES

[1] David E. Bernal, Kyle E. C. Booth, Raouf Dridi, Hedayat Alghassi, Sridhar Tayur, and Davide Venturelli. 2020. Integer Programming Techniques for Minor-Embedding in Quantum Annealers. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. arXiv:1912.08314.

[2] Kelly Boothby, Paul Bunyk, Jack Raymond, and Aidan Roy. 2020. Next-Generation Topology of D-Wave Quantum Processors. arXiv:2003.00133 [quant-ph] arXiv:2003.00133.

[3] Kelly Boothby, Andrew D. King, and Jack Raymond. 2021. *Zephyr Topology of D-Wave Quantum Processors.* Technical Report. D-Wave Systems.

[4] Tomas Boothby, Andrew D. King, and Aidan Roy. 2016. Fast Clique Minor Generation in Chimera Qubit Connectivity Graphs. *Quantum Information Processing* 15 (2016), 495–508.

[5] Jun Cai, William G. Macready, and Aidan Roy. 2014. A Practical Heuristic for Finding Graph Minors. arXiv:1406.2741 [quant-ph] arXiv:1406.2741.

[6] Raouf Dridi, Hedayat Alghassi, and Sridhar Tayur. 2018. A Novel Algebraic Geometry Compiling Framework for Adiabatic Quantum Computations. arXiv:1810.01440 [quant-ph] arXiv:1810.01440.

[7] Pablo Gómez-Tejedor, Eneko Osaba, and Esther Villar-Rodriguez. 2025. Addressing the Minor-Embedding Problem in Quantum Annealing and Evaluating State-of-the-Art Algorithm Performance. arXiv:2504.13376 [quant-ph] arXiv:2504.13376.

Table 9. Average chain length, mean (std) over 5 seeds, for all 35 instance–topology cells. Columns: `minorminer`, `minorminer-layout`, `pathfinder`, `pathfinder-thorough`.

cell	mm	mm-lay	pathf	pf-thor
Pegasus P_6 (clean)				
ER n=20, d=0.3	1.79 (0.09)	1.76 (0.05)	1.79 (0.09)	1.65 (0.03)
ER n=20, d=0.5	2.28 (0.12)	2.34 (0.06)	2.25 (0.07)	2.19 (0.04)
ER n=20, d=0.7	2.59 (0.08)	2.55 (0.09)	2.57 (0.09)	2.51 (0.07)
ER n=30, d=0.3	2.70 (0.11)	2.75 (0.15)	2.69 (0.10)	2.59 (0.03)
ER n=30, d=0.5	3.30 (0.16)	3.43 (0.10)	3.25 (0.15)	3.19 (0.10)
ER n=30, d=0.7	3.83 (0.22)	3.80 (0.17)	3.79 (0.23)	3.65 (0.06)
ER n=40, d=0.3	3.56 (0.05)	3.62 (0.14)	3.52 (0.06)	3.41 (0.06)
ER n=40, d=0.5	4.57 (0.10)	4.72 (0.13)	4.49 (0.14)	4.38 (0.08)
ER n=40, d=0.7	5.15 (0.06)	5.23 (0.17)	5.07 (0.04)	4.97 (0.04)
reg n=30, k=4	1.37 (0.06)	1.41 (0.11)	1.37 (0.06)	1.31 (0.03)
reg n=30, k=6	1.92 (0.11)	1.94 (0.04)	1.92 (0.11)	1.86 (0.07)
reg n=40, k=4	1.51 (0.12)	1.48 (0.06)	1.51 (0.11)	1.35 (0.04)
reg n=40, k=6	2.10 (0.13)	2.13 (0.23)	2.09 (0.12)	1.95 (0.03)
BA n=30, m=3	1.56 (0.11)	1.51 (0.07)	1.56 (0.11)	1.50 (0.06)
BA n=30, m=5	2.22 (0.09)	2.25 (0.12)	2.20 (0.09)	2.09 (0.07)
BA n=40, m=3	1.80 (0.06)	1.73 (0.08)	1.79 (0.06)	1.71 (0.09)
BA n=40, m=5	2.56 (0.10)	2.63 (0.13)	2.50 (0.08)	2.44 (0.03)
broken Pegasus P_6 (5% faults)				
ER n=20, d=0.3	1.85 (0.12)	1.90 (0.15)	1.84 (0.12)	1.70 (0.03)
ER n=20, d=0.5	2.26 (0.07)	2.35 (0.10)	2.25 (0.06)	2.23 (0.05)
ER n=20, d=0.7	2.73 (0.14)	2.69 (0.16)	2.72 (0.14)	2.56 (0.09)
ER n=30, d=0.3	2.79 (0.12)	2.79 (0.12)	2.79 (0.13)	2.67 (0.07)
ER n=30, d=0.5	3.53 (0.11)	3.50 (0.10)	3.48 (0.13)	3.34 (0.09)
ER n=30, d=0.7	3.85 (0.17)	4.01 (0.16)	3.79 (0.14)	3.68 (0.13)
ER n=40, d=0.3	3.62 (0.15)	3.63 (0.19)	3.58 (0.10)	3.52 (0.09)
ER n=40, d=0.5	4.88 (0.18)	4.71 (0.19)	4.80 (0.18)	4.52 (0.06)
ER n=40, d=0.7	5.52 (0.19)	5.36 (0.20)	5.34 (0.18)	5.08 (0.18)
Zephyr Z_4				
ER n=20, d=0.3	1.53 (0.02)	1.55 (0.04)	1.53 (0.02)	1.45 (0.03)
ER n=20, d=0.5	1.96 (0.04)	1.94 (0.07)	1.96 (0.04)	1.83 (0.02)
ER n=20, d=0.7	2.16 (0.07)	2.35 (0.16)	2.16 (0.07)	2.11 (0.04)
ER n=30, d=0.3	2.24 (0.04)	2.28 (0.08)	2.23 (0.06)	2.18 (0.09)
ER n=30, d=0.5	2.75 (0.07)	2.73 (0.02)	2.71 (0.07)	2.69 (0.06)
ER n=30, d=0.7	3.03 (0.06)	3.03 (0.04)	3.01 (0.05)	2.96 (0.03)
ER n=40, d=0.3	2.92 (0.09)	3.11 (0.13)	2.88 (0.09)	2.83 (0.11)
ER n=40, d=0.5	3.63 (0.08)	3.60 (0.16)	3.56 (0.07)	3.52 (0.03)
ER n=40, d=0.7	4.18 (0.30)	4.29 (0.06)	4.09 (0.26)	3.85 (0.07)

[8] Timothy D. Goodrich, Blair D. Sullivan, and Travis S. Humble. 2018. Optimizing Adiabatic Quantum Program Compilation Using a Graph-Theoretic Framework. *Quantum Information Processing* 17 (2018).

[9] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1706.02216.

[10] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

[11] Diederik P. Kingma and Max Welling. 2014. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations (ICLR)*. arXiv:1312.6114.

[12] Thomas N. Kipf and Max Welling. 2016. Variational Graph Auto-Encoders. *NeurIPS Bayesian Deep Learning Workshop* (2016). arXiv:1611.07308.
[13] Andrew Lucas. 2014. Ising Formulations of Many NP Problems. *Frontiers in Physics* 2 (2014), 5.
[14] Larry McMurchie and Carl Ebeling. 1995. PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs. In *ACM International Symposium on Field-Programmable Gate Arrays (FPGA)*. 111–117.
[15] Kurt Mehlhorn. 1988. A Faster Approximation Algorithm for the Steiner Problem in Graphs. *Inform. Process. Lett.* 27, 3 (1988), 125–128.
[16] Riccardo Nembrini, Maurizio Ferrari Dacrema, and Paolo Cremonesi. 2025. Minor Embedding for Quantum Annealing with Reinforcement Learning. arXiv:2507.16004 [quant-ph] arXiv:2507.16004.
[17] Hoang M. Ngo, Tu N. Do, Duc Vu, Tamer Kahveci, and My T. Thai. 2024. CHARME: A Chain-Based Reinforcement Learning Approach for the Minor Embedding Problem. arXiv:2406.07124 [quant-ph] arXiv:2406.07124.
[18] Elijah Pelofske. 2024. 4-Clique Network Minor Embedding for Quantum Annealers. *Physical Review Applied* 21 (2024), 034023. arXiv:2301.08807.
[19] José P. Pinilla and Steven J. E. Wilton. 2019. Layout-Aware Embedding for Quantum Annealing Processors. In *ISC High Performance*.
[20] Martin Simonovsky and Nikos Komodakis. 2018. GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders. In *International Conference on Artificial Neural Networks (ICANN)*. arXiv:1802.03480.
[21] Stefano Sinno, Thomas Groß, Nicholas Chancellor, et al. 2025. Optimised Quantum Embedding via Complete Bipartite Graphs. arXiv:2504.21112 [quant-ph] arXiv:2504.21112.
[22] Yuya Sugie, Yuki Yoshida, Nozomu Mukai, Hiroshi Banno, Hiroyuki Shioda, et al. 2018. Graph Minors from Simulated Annealing for Annealing Machines with Sparse Connectivity. In *Theory and Practice of Natural Computing (TPNC)*.
[23] Hiromitsu Takahashi and Akira Matsuyama. 1980. An Approximate Solution for the Steiner Problem in Graphs. In *Mathematica Japonica*, Vol. 24. 573–577.
[24] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*. arXiv:1710.10903.
[25] Cédric Vincent-Cuaz, Rémi Flamary, Marco Corneli, Titouan Vayer, and Nicolas Courty. 2022. Semi-Relaxed Gromov-Wasserstein Divergence and Applications on Graphs. In *International Conference on Learning Representations (ICLR)*.
[26] Hongteng Xu, Dixin Luo, Hongyuan Zha, and Lawrence Carin. 2019. Gromov-Wasserstein Learning for Graph Matching and Node Embedding. In *International Conference on Machine Learning (ICML)*. arXiv:1901.06003.
[27] Stefanie Zbinden, Andreas Bärtschi, Hristo Djidjev, and Stephan Eidenbenz. 2020. Embedding Algorithms for Quantum Annealers with Chimera and Pegasus Connection Topologies. In *ISC High Performance*.